

3.2 Strings

Sue has just received an e-mail from her Grandma Ruth. Grandma, unfortunately, is new to computers (and typing for that matter) and has written the entire message using only capital characters. This is very difficult to read! Rather than suffer through the entire message, she writes a program to convert the all-caps text to sentence case.

Objectives

By the end of this class, you will be able to:

- Understand the role the null character plays in string definitions.
- Write a loop to traverse a string.

Prerequisites

Before reading this section, please make sure you are able to:

- Declare an array to solve a problem (Chapter 3.0).
- Write a loop to traverse an array (Chapter 3.0).
- Predict the output of a code fragment containing an array (Chapter 3.0).
- Pass an array to a function (Chapter 3.0).

Overview

While we have been using text (`char text[256];`) the entire semester, we have yet to discuss how it works. Through this chapter, we will learn that strings have a little secret: they are just arrays. Specifically, they are arrays of characters with a null-character at the end.

```
{
    char text[] =
    {
        'C', 'S', ' ', '1', '2', '4', '\0'
    };
    cout << text << endl;
}
```

One common task to perform on strings is to write a loop to traverse them. Consider the following loop prompting the user for text and displaying it one letter at a time on the screen:

```
{
    char text[256];           // strings are arrays of characters
    cout << "Enter text: ";   // using the double quotes creates a string literal
    cin >> text;              // CIN puts the null-character at the end of strings

    for (int i = 0; text[i]; i++) // almost our second standard FOR loop
        cout << text[i] << endl; // we can access strings one character at a time
}
```

This chapter will discuss why strings are defined as arrays of characters with a null-character, how to declare a string and pass one to a function, and how to traverse a string using a FOR loop.

Implementation of Strings

An integer is a singular value that can always fit into a single 4-byte block of memory. Text, however, is fundamentally different. Text can be any length, from a few letters to a complete novel. Because text is an arbitrary length, it cannot be stuck into a single location of memory as a number can. It is therefore necessary to use a different data representation: an array.

Text is fundamentally a one-dimensional construct. Each letter in a book can be uniquely addressed from its offset (index) from the beginning of the manuscript. For this reason, text is fundamentally an array of characters. We call these arrays of characters “**strings**” because they are “strings of characters” somewhat like a collection of characters attached by a string.

I	n		t	h	e		b	e	g	i	n	n	i	n	g		...
---	---	--	---	---	---	--	---	---	---	---	---	---	---	---	---	--	-----

Because text can be a wide variety of lengths, it is necessary to have some indicator to designate its size. There are two fundamental strategies: keep an integral variable to specify the length, or specify an end-of-string marker. The first method is called **Pascal-strings** because the programming language Pascal uses it as the default string type. This method specifies that the first character of the string is the length:

6	C	S		1	2	4
---	---	---	--	---	---	---

The second method, called **c-strings**, puts an end-of-string marker. This marker is called the null-character:

C	S		1	2	4	null
---	---	--	---	---	---	------

While either design is viable, c-strings are used exclusively in C++ and are the dominant design in programming languages today. The main reason for this is that each slot in a character array is, well, a character. This means that the maximum value that can be put in a character slot is 255. Thus, the maximum length of a Pascal-string is 255 characters. C-strings do not have this limitation; they can be any length.

Null-character

The null-character (`'\0'`) is a special character used to designate the end of a c-string. We can assign the null-character to any char variable:

```
{
    char nullCharacter = '\0';           // single character assigned a null

    char text[256];                     // text is just an array of characters
    text[0] = '\0';                     // putting the null at the beginning signifies
}                                       // an empty string
```

The value of the null-character is 0 on the ASCII Table:

```
assert('\0' == 0);
```

There is a special reason why the null-character was given the first slot on the ASCII table: it is the only character that equates to `false` when cast to a `bool`. In other words:

```
assert('\0' == false);
```

Since zero is the only integer mapping to `false`, we can assume that the null-character is the only `false` character in the ASCII table.

C-Strings

All strings are c-strings by default in C++. To illustrate, consider the following code:

```
cout << "Hello world" << endl;
```

One may think that this string uses eleven bytes of memory (the space character is also a character). However, it actually takes twelve:

H	e	l	l	o		w	o	r	l	d	null
---	---	---	---	---	--	---	---	---	---	---	------

We can verify this with the following code:

```
cout << sizeof("Hello world") << endl;
```

The output, of course, will be 12.

String Syntax

In all ways, we can treat strings as simple arrays. There are two characteristics of strings, however, which make them special. The first characteristic is the existence of the null-character. While all strings are character arrays, not all character arrays are strings. In other words, an array of letter grades for a class will probably not be a string because there is no null-character. The existence of the null eliminates the need to pass a length parameter when calling a function. However, we will still probably want to pass a buffer size variable.

The second characteristic is the double-quotes notation. Consider the following two strings with equivalent declarations:

```
{
    char text1[] =
    {
        'C', 'S', ' ', '1', '2', '4', '\0'    // the hard way. Don't do this!
    };
    char text2[] = "CS 124";                // ah, much better...
}
```

Clearly the double-quote notation greatly simplifies the declaration of strings. Whenever we see these double-quotes, however, we must always remember the existence of the hidden null-character.

Declaring a string

The rules for declaring a string are very similar to those of declaring an array:

Declaration	In memory	Description										
char text[10];	<table><tr><td>?</td><td>?</td><td>?</td><td>?</td><td>?</td><td>?</td><td>?</td><td>?</td><td>?</td><td>?</td></tr></table>	?	?	?	?	?	?	?	?	?	?	None of the slots are initialized
?	?	?	?	?	?	?	?	?	?			
char text[7] = "CS 124";	<table><tr><td>C</td><td>S</td><td></td><td>1</td><td>2</td><td>4</td><td>\0</td></tr></table>	C	S		1	2	4	\0	The initialized size is the same as the declared size			
C	S		1	2	4	\0						
char text[10] = "CS 124";	<table><tr><td>C</td><td>S</td><td></td><td>1</td><td>2</td><td>4</td><td>\0</td><td>\0</td><td>\0</td><td>\0</td></tr></table>	C	S		1	2	4	\0	\0	\0	\0	The first 7 are initialized, the balance are filled with 0
C	S		1	2	4	\0	\0	\0	\0			
char text[] = "CS 124";	<table><tr><td>C</td><td>S</td><td></td><td>1</td><td>2</td><td>4</td><td>\0</td></tr></table>	C	S		1	2	4	\0	Declared to exactly the size necessary to fit the text			
C	S		1	2	4	\0						



Sue's Tips

A common mistake when declaring a string is to fail to leave space for the null character. Remember that the size of the declared string must be one greater than the maximum number of characters that could possibly be in the string.

Passing a string

Since strings are just arrays, exactly the same rules apply to them as they apply to arrays. This means that they can always be treated like pass-by-reference parameters. Consider the following function:

```
/* *****  
 * GET NAME  
 * Prompt the user for his first name  
 * ***** */  
void getName(char name[])  
{  
    cout << "What is your first name? ";  
    cin >> name;  
}
```

Observe how there is no return statement or ampersand (&) on the name parameter. Because we are passing an array, we still get to fill the value. Now, consider the following code to call this function:

```
{  
    getName("String literal");    // ERROR! There is no variable passed here!  
  
    char name[256];  
    getName(name);                // this works, a variable was passed  
}
```

What went wrong? The function asked for an array of characters and the caller provided it! The answer to this conundrum is a bit subtle.

A string literal (such as "String literal" above) refers to data in the read-only part of memory. This data is not associated with any variable and cannot be changed. The data-type is therefore not an "array of characters," but rather a "constant array of characters." In other words, it cannot be changed.

When authoring a function where the input text is treated as a read-only value, it is very important to add a `const` prefix. This prefix enables the caller of the function to pass a string literal instead of a string variable:

```
/* *****  
 * Determine if a given file exists  
 * ***** */  
bool checkFile(const char fileName[]) // CONST modifier allows caller to  
{ // pass a literal  
    ifstream fin(fileName);  
  
    if (fin.fail())  
        return false;  
  
    fin.close();  
    return true;  
}
```

Now, due to the `const` modifier, either of the following is valid:

```
{  
    char fileName[] = "data.txt"; // string variable that is read/write  
    checkFile(fileName); // the CONST modifier has no effect here  
  
    checkFile("data.txt"); // this would be a compile error without  
} // the CONST modifier in checkFile()
```

Comparing Strings

Recall that strings are arrays of characters terminated with a null character. Consider two arrays of integers. The only way to compare if two lists of numbers are the same is to compare the individual members.

```
{  
    int list1[] = { 4, 8, 12 };  
    int list2[] = { 3, 6, 9 };  
    if (list1 == list2) // this compares the location of the  
        cout << "Same!"; // two lists in memory. It does _NOT_  
    else // compare the contents of the lists!  
        cout << "Different!";  
}
```

Instead, a loop is required:

```
{  
    int list1[] = { 4, 8, 12 };  
    int list2[] = { 3, 6, 9 };  
    bool same = true;  
    for (int i = 0; i < 3; i++) // we must go through each item in the  
        if (list1[i] != list2[i]) // two lists to see if they are the same.  
            same = false; // There is no other way!  
}
```

The only way to see if the lists are the same is to write a loop. Similarly, c-strings cannot be compared with a single `==` operator. To compare two strings, it is necessary to write a loop to traverse the strings!

```
{  
    char text1[256] = "Computer Science";  
    char text2[256] = "Electrical Engineering";  
    if (text1 == text2) // this will _NOT_ compare  
        cout << "Same!"; // the contents of the two  
    else // strings. It will only  
        cout << "Different!"; // compare the addresses!  
}
```

Traversing a String

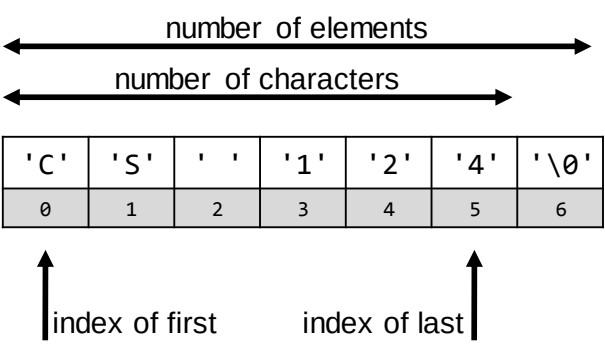
Strings are just arrays of characters where the end is marked with the null character ('\\0'). This means we can use a FOR loop to walk through a string much the same way we do with an integer array. The only difference is how we know we have reached the end of the array. With arrays of numbers, we typically need to pass another parameter (ex: numElements) to tell us when we are at the end:

```
{
    for (int i = 0; i < numElements; i++)
        cout << array[i] << endl;
}
```

So, how big is the string? Consider the following string:

```
char name[] = "CS 124";
```

This corresponds to the following layout in memory:



From here, we have the following relationships:

Amount of memory used	sizeof(name)
Number of elements in the array	sizeof(name) / sizeof(name[0])
Number of characters in the string (minus null)	(sizeof(name) / sizeof(name[0]) - 1)
Index of the last item	(sizeof(name) / sizeof(name[0]) - 2)

Therefore, to walk through an array backwards, you will need:

```
{
    char name[] = "CS 124";
    for (int i = (sizeof(name) / sizeof(name[0]) - 2); i >= 0; i--)
        cout << name[i] << endl;
}
```

It is important to remember that this only works when the size of the buffer is the same as the size of the string. This is not commonly the case!

Traversing using the null-character

With strings, we do not typically have a variable indicating the length of the string. Instead, we look for the null character:

```
{
    for (int i = 0; text[i] != '\0'; i++)
        cout << text[i];
}
```

In this case, we are going to keep iterating until the null-character (`'\0'`) is encountered:

0	1	2	3	4	5	6
C	S		1	2	4	\0

Note that the null character (`'\0'`) has the ASCII value of zero. This is convenient because, when it is casted to a `bool`, it is the only `false` value in the ASCII table. Therefore, we can loop through a string with the following code:

```
{
    for (int i = 0; text[i]; i++)
        cout << text[i];
}
```

Observe how the condition in the FOR loop checks if the i^{th} character in the string is true or, in other words, not the null-character.

0	1	2	3	4	5	6
C	S		1	2	4	\0
true	true	true	true	true	true	false

Consider the case where a string of text exists with spaces between the words. The problem is to convert the spaces to underscores. In other words “Introduction to Software Development” becomes “Introduction_to_Software_Development”:

```
{
    char text[] = "Software Development";           // target string, any will do
    for (int i = 0; text[i]; i++)                   // loop through all items in the string
        if (text[i] == ' ')                         // check each item against a space
            text[i] = '_';                           // replace with an underscore
}
```

Example 3.2 – Toggle Case

Demo

This example will demonstrate how to walk through a string, modifying each character one at a time. This will be done by using an index to loop through the string until the null character is encountered.

Problem

Consider the scenario where the unfortunate user typed his entire e-mail message with the CAPS key on. This will not only capitalize most characters, but it will un-capitalize the first letter of each sentence. Write a function to correct this error.

Please enter your text: **SOFTWARE eENGINEERING**
Software Engineering

Solution

The function to convert the text will take a string as a parameter. Recall that arrays are always pass-by-reference. This means that the parameter can serve both as the input and output of the function.

```
void convert(char text[]);
```

To traverse the string we will loop through each item with an index. Unlike with a standard array, we need to use the condition `text[i]`, not `i < num`:

```
for (int i = 0; text[i]; i++)  
    ;
```

With this loop, we can look at every character in the string. Now we will need to determine if a character is uppercase or lowercase. If the character is uppercase (`isupper()`), then we need to lowercase it (`tolower()`). Otherwise, we need to uppercase it (`toupper()`). Note that the functions `isupper()`, `tolower()`, and `toupper()` are in the `cctype` library

```
/******  
 * CONVERT  
 * Convert uppercase to lowercase and vice versa  
 *****/  
void convert(char text[])  
{  
    for (int i = 0; text[i]; i++)        // loop through all the elements in text  
        if (isupper(text[i]))           // check each element's case  
            text[i] = tolower(text[i]);  // convert to lower if it is upper  
        else  
            text[i] = toupper(text[i]);   // otherwise convert to upper  
}
```

Challenge

As a challenge, try to modify the above program to count the number of uppercase letters in the input stream. The prototype is:

```
int countUpper(const char text[]);
```

See Also

The complete solution is available at [3-2-toggleCase.cpp](#) or:

`/home/cs124/examples/3-2-toggleCase.cpp`



Example 3.2 – Sentence Case

Demo	This example will demonstrate how to traverse a string, processing each character individually.
Problem	Write a program to to perform a sentence-case conversion (where only the first letter of a sentence is capitalized) on an input string. Please enter your input text: <u>this IS SOME tExT. soME More TeXt.</u> The sentence-case version of the input text is: This is some text. Some more text.
Solution	Sentence-casing is the process of converting every character to lower-case except the first character in the sentence. In other words, after a period, exclamation point, or question mark is encountered, the next letter needs to be uppercase. We capture this condition with the <code>isNewSentence</code> variable. If a sentence-ending character is found, we set <code>isNewSentence</code>. This variable remains set until an alphabetic character is found (<code>isalpha()</code>). In this case, we convert the letter to uppercase (<code>toupper()</code>) and set <code>isNewSentence</code> to false. Otherwise, we set the letter to lowercase (<code>tolower()</code>). <pre> void convert(char text[]) { // the first letter of the input is the start of a sentence bool isNewSentence = true; // traverse the string for (int i = 0; text[i]; i++) { if (text[i] == '.' text[i] == '!' text[i] == '?') isNewSentence = true; // convert the first letter to uppercase if (isNewSentence && isalpha(text[i])) { text[i] = toupper(text[i]); isNewSentence = false; } // everything else to lowercase else text[i] = tolower(text[i]); } } </pre>
Challenge	As a challenge, modify the above program to handle Title Case the text. This is done by converting every character to lowercase except the first letter of the word. In other words, the first letter after every space is uppercase (as opposed to the first letter after every sentence-ending punctuation).
See Also	The complete solution is available at 3-2-sentenceCase.cpp or: /home/cs124/examples/3-2-sentenceCase.cpp 

Example 3.2 – Backwards String

Demo	This example will demonstrate how to traverse a string backwards. This will require two loops through the string: forward to find the end of the string, and reverse to display the string backwards.
Problem	Write a program to prompt the user for some text and display the text backwards: <pre>Please enter some text: Software Engineering The text backwards is "gnireenignE erawtfoS"</pre>
Solution	This function will consist of two parts: looping through the string to find the null-character, then progressing backwards to display the contents of the string. <pre> /***** * DISPLAY BACKWARDS * Display a string backwards *****/ void displayBackwards(const char text[]) // const for string literals { // first find the end of the string int i = 0; // needs to be a local variable while (text[i]) // as long as the null is not found i++; // keep going through the string // now go backwards for (i--; i >= 0; i--) // back up one because we went too far cout << text[i] // display each individual character cout << endl; } </pre> When we are finished with the first loop (the forward moving one), the index will be referring to the location of the null-character. We don't want to display the null-character. Therefore, the first step of the display loop is to back the index by one slot. From here, it displays each character including first character of the string (with the 0 index).
Challenge	Notice how a WHILE loop is used instead of a FOR loop in the first part of the function. As a challenge try to re-write this loop as a standard FOR loop. Again, notice how the second loop uses a FOR loop. Can you re-write it to use a WHILE loop instead?
See Also	The complete solution is available at 3-2-backwards.cpp or: <pre>/home/cs124/examples/3-2-backwards.cpp</pre> 

Problem 1

How much memory is reserved by each of the following?

char text[8]

char text[] = "CS 124";

char text[10] = "124";

Please see page 244 for a hint.

Problem 2

What is the output of the following code?

```
{
    char text1[] = "Text";
    char text2[] = "Text";

    if (text1 == text2)
        cout << "Equal";
    else
        cout << "Different"
}
```

Answer:

Please see page 246 for a hint.

Problem 3

What is the output of the following code?

```
char text1[] = "this";
char text2[] = "that";

if (text1[0] == text2[0])
    cout << "Equal";
else
    cout << "Different";
}
```

Answer:

Please see page 246 for a hint.

Problem 4

What is the output of the following code?

```
{
    char text[5] = "42";
    cout << text[4] << endl;
}
```

Answer:

Please see page 244 for a hint.

Problem 5

Which of the following is the correct prototype of a string passed to a function as a parameter?

`void displayText(char text);`

`void displayText(char text []);`

`void displayText(text);`

`void displayText(char [] text);`

Please see page 221 for a hint.

Problem 6

Consider the following function prototype from #5:

How would you call the function `displayText();`?

Answer:

Please see page 221 for a hint.

Problem 7

What is the output of the following code?

```
{
    char text[] = "Hello";

    for (int i = 0; i < 6; i++)
    {
        bool value = text[i];
        cout << value;
    }

    cout << endl;
}
```

Answer:

Please see page 243 for a hint.

Assignment 3.2

Traversing a string (or any other type of array for that matter) is a common programming task. This assignment is the first part in a two-part series (the other is Assignment 3.5) where we will learn different techniques for visiting every member of a string. Your assignment is to write the function `countLetters()` then a driver program to test it.

countLetters

Write a function to return the number of letters in a string. This involves traversing the string using the array notation (with an index as we have been doing all semester). We will re-write this function in Assignment 3.5 to do the same thing using a pointer.

Driver Program

Create a `main()` that prompts the user for a line of input (using `getline`), calls `countLetters()`, and displays the number of letters.

Note that since the first `cin` will leave the stream pointer on the newline character, you will need to use `cin.ignore()` before `getline()` to properly fetch the section input. Take the first example below, the input buffer will look like:

z	\n	N	o	Z	'	s	H	e	r	e	!	\n
---	----	---	---	---	---	---	---	---	---	---	---	----

If the program inputs a character followed by a line, the code will look like this:

```
cin >> letter;  
cin.getline(text, 256);
```

After the first `cin`, the input pointer will point to the 'z'. When the `getline` statement gets executed next, it will accept all input up to the next newline character. Since the pointer is already on the newline character, the result will be an empty string. To skip this newline character, we use `cin.ignore()`.

Example

Two examples... The user input is underlined.

Example 1:

```
Enter a letter: z  
Enter text: NoZ'sHere!  
Number of 'z's: 0
```

Example 2:

```
Enter a letter: a  
Enter text: Brigham Young University - Idaho  
Number of 'a's: 2
```

Assignment

The test bed is available at:

```
testBed cs124/assign32 assignment32.cpp
```

Don't forget to submit your assignment with the name "Assignment 32" in the header.

Please see pages 40 and 249 for a hint.